Hierarchical Deep Reinforcement Learning-Based Partial Task Offloading in Device-Edge-Cloud Computing Architecture

Ihsan Ullah, Hyun-Kyo Lim, Yeong-Jun Seok, and Youn-Hee Han, *Member, IEEE*

***Abstract*—The rapid growth of the Intelligent Internet of Things (IIoT) and 5G applications demands high-performance computing, low latency, and energy efficiency, often beyond the capacity of local devices. Task offloading to edge or cloud servers offers a promising solution. However, achieving optimal partial task distribution across device, edge, and cloud remains challenging due to dynamic network conditions, heterogeneous resources, and conflicting performance objectives. This paper presents a Hierarchical Deep Reinforcement Learning (HDRL) framework for intelligent, resource-aware, latency-constrained partial task offloading in device–edge–cloud architectures. The problem is formulated as a two-level Markov Decision Process (MDP) for structured decision-making. At the high level, a Double Deep Q-Network (DDQN) agent observes the global network state and selects computation tiers. At the low level, a Proximal Policy Optimization (PPO) agent determines optimal task-splitting ratios, enabling fine-grained control of subtask distribution. The HDRL framework jointly optimizes energy consumption, task acceptance ratio, and resource utilization while ensuring end-to-end latency constraints. The system adapts dynamically to varying workloads and resource availability. Simulations showed that HDRL-PTO achieved 80–90% task acceptance, reduced energy consumption compared to DRL-PTO and DRL-TO, and consistently delivered higher episode rewards across Network I and II.**

***Index Terms*—Edge-cloud computing, partial task offloading, hierarchical deep reinforcement learning, Markov decision process, resource allocation, deep reinforcement learning.**

I. Introduction

NEW APPLICATIONS enabled by IoT and 5G/6G, such as surveillance, AR/VR, and face recognition demand substantial computing power. While 5G/6G provide higher bandwidth, connection density, and lower latency, IoT devices have limited processing capabilities for resource-intensive tasks. Offloading tasks to cloud servers is common but limited by latency and data transfer delays, making it unsuitable for real-time use. Edge computing addresses this by placing computational resources near IoT devices, allowing secure, reliable, real-time task offloading to edge servers instead of the cloud [1]. Traditional cloud computing offloads tasks from end devices to remote cloud servers (CCSs), but growing devices and workloads increase traffic, congestion, latency, and degrade quality of service (QoS). Edge computing mitigates this by deploying resources near IoT devices [2], [3], enabling real-time, secure processing for delay-sensitive tasks. Due to limited capacity and workload variability of edge servers (ECSs), horizontal cooperation among ECSs balances load and improves utilization [4], while vertical collaboration with CCSs leverages cloud resources to reduce bottlenecks and latency [5], [6], [7], [8], [9]. Partial task offloading divides tasks for local and edge/cloud execution [10], optimizing energy efficiency and task acceptance. Yet, finding the optimal split is complex, depending on offloading location, compute time, resource availability, energy use, and network conditions.

Deep reinforcement learning (DRL) is a powerful AI method for controlling complex dynamic systems, especially with high-dimensional state spaces. It has been applied to task and resource allocation [11], but struggles with effective decision-making in complex task offloading. Leveraging DRL for partial task offloading to maximize bandwidth use and reduce energy remains a challenge. Hierarchical deep reinforcement learning (HDRL) addresses this by decomposing tasks into simpler subtasks with a hierarchical structure, reducing computational complexity and enabling reusable learning. Prior studies show HDRL is promising for complex DRL problems [12].

This paper proposes Hierarchical Deep Reinforcement Learning for Partial Task Offloading (HDRL-PTO), an intelligent offloading scheme for three-tier (device, edge, cloud) environments. It balances resource use by partially offloading tasks while minimizing energy consumption and maximizing tasks meeting latency constraints. HDRL reduces offloading complexity and speeds up learning. HDRL-PTO has two agents: a high-level (HL) agent using Double DQN to select computation locations (local, edge, cloud), and a low-level (LL) agent using PPO to decide task splitting ratios for the chosen locations. The HL agent may offload tasks entirely to one location or split across two or three locations, delegating splitting decisions to the LL agent. We formulate the problem

Received 20 January 2025; revised 24 August 2025; accepted 13 September 2025. Date of current version 25 March 2026. This work was supported in part by the National Science and Technology Council of Taiwan under Grant NSTC113-2222-E-155-007 and Grant NSTC114-2221-E-155-021, and in part by the National Research Foundation of Korea (NRF) funded by the Ministry of Education under Grant NRF-2023R1A2C1003143 and Grant NRF-2018R1A6A1A03025526. *(Corresponding authors: Ihsan Ullah and Youn-Hee Han.)*

Ihsan Ullah is with the Department of Computer Science and Engineering, Yuan Ze University, Taoyuan 320315, Taiwan (e-mail: ihsan@saturn.yzu.edu.tw).

Hyun-Kyo Lim, Yeong-Jun Seok, and Youn-Hee Han are with the Future Convergence Engineering, Korea University of Technology and Education, Cheonan 31253, South Korea (e-mail: glenn89@koreatech.ac.kr; dsb04163@koreatech.ac.kr; yhhan@koreatech.ac.kr).

Digital Object Identifier 10.1109/TCE.2025.3611973

1558-4127 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

as a multi-objective Markov Decision Process (MDP) aiming to minimize energy consumption and maximize task completion within latency limits. The proposed HDRL algorithms leverage this hierarchical MDP structure. Simulations show HDRL-PTO outperforms other DRL methods in resource utilization and energy efficiency. The main contributions are summarized below:

- The proposed HDRL architecture optimizes partial task offloading in device-edge-cloud environments, balancing task distribution while minimizing energy use and maximizing timely completion.
- It uses a hierarchical approach: the high-level Double DQN agent selects tasks and computation levels, while the low-level PPO agent decides offloading ratios for each selected level.
- This study is the first to apply hierarchical DRL to partial task offloading in device-edge-cloud architectures.
- HDRL-PTO outperforms existing DRL methods in energy efficiency, task acceptance, and adaptability to dynamic, diverse tasks.

The paper is organized as follows: Section II reviews related work; Section III introduces the system model; Section IV presents the problem formulation and hierarchical MDPs; Section V details policy learning for partial task offloading; Section VI shows experimental results; Section VII concludes.

II. Related Works

A. Traditional Optimization Algorithm-Based Solutions

Traditional optimization methods have been widely used for joint energy, latency, and resource optimization in multi-tier MEC and edge-cloud systems. For example, Younis et al. [13] combined task offloading with approximate computing using dual decomposition to balance energy and latency. Xu et al. [14] and Chen et al. [6] employed game-theoretic approaches for resource optimization in NOMA-based vehicular edge computing and device-edge-cloud systems, respectively. Multi-stage frameworks minimizing latency were developed using MINLP formulations [15], [16], while hybrid genetic algorithms have been proposed for offloading in cooperative edge-cloud computing [17]. Energy-efficient and latency-aware offloading in industrial cloud-edge environments was addressed by Zhai et al. [18] through joint optimization of task time and energy via IT-MOEA. Several works applied distributed and decentralized methods: collaborative cloud-edge computing with wireless backhaul reducing transmission delay [19], Lyapunov optimization for green parallel offloading in IoT-edge tasks [20], and decentralized task offloading in OFDMA heterogeneous networks minimizing energy-delay costs [21]. Robust and risk-aware task offloading models were proposed to handle delay uncertainties [22], and calibrated bandit learning was used for decentralized offloading in ultra-dense MEC networks [23].

B. DRL Algorithm-Based Solutions

1) Value-Based Methods: Wu et al. [24] combined DRL and federated learning for virtual network embedding to enhance resource management with indivisible tasks. Lu et al. [25] used DQN for large-scale heterogeneous MEC without prior environment knowledge. Huang et al. [26] and [27] optimized task offloading and bandwidth allocation using DQN and hybrid DRL under QoS, latency, and energy constraints in MEC and D2D-assisted networks. Multi-hop offloading relying on centralized SDN control was studied in [28], while cloud-edge task offloading combining multi-round resource auctions with Double DQN and Dueling DQN was proposed in [29] to optimize edge resources under capacity and latency limits. Deep Meta Q-Learning improved multi-task offloading adaptability in edge-cloud systems [30]. Migration optimization via multi-agent RL appeared in [31], and Chen et al. [32] combined double DQN, distributed LSTM, and intermediate state transitions in a game-theoretic algorithm to reduce offloading complexity in dynamic conditions.

2) Policy-Based Methods: Chen and Wu [33] proposed two DRL algorithms to reduce service latency in edge computing. Qiu et al. [34] developed a distributed DRL approach for computation offloading in deadline-sensitive, resource-intensive applications. Joint multi-task offloading and resource allocation for satellite IoT was studied in [35]. Actor-critic methods, including asynchronous A2C [36] and DDPG [37], were applied to optimize resource allocation, energy consumption, and wireless power transfer in multiuser and digital twin networks. Zou et al. [38] examined partial offloading schemes for latency reduction, while Liu et al. [39] formulated semi-Markov optimization for vehicular mobile edge servers. Partial task offloading in edge-cloud environments was also explored in [10]. Reference [40] utilized a Monte Carlo–based policy DRL method to enhance task offloading efficiency in edge and fog computing.

Jiang et al. [41] proposed an HDRL framework for joint computation offloading and resource allocation in device-to-device communication, focusing on device-to-device and device-to-edge offloading without involving the cloud. Motivated by their work, we propose a novel HDRL method for partial task offloading in device-edge-cloud networks, addressing the increased complexity from more devices, edge servers, and cloud integration.

III. System Model

End devices connect wirelessly to base stations (BS), which link to cloud servers via wired backhaul. Devices can process tasks locally or offload them to edge or cloud servers. We propose dividing tasks into three portions by a specified ratio, distributing them across local, edge, and cloud levels based on available resources. Our scheme includes network, task, communication, delay, and energy models, detailed in the following sections. Table I lists the mathematical notations used by the proposed models.

A. Network Model

Fig. 1 shows the system architecture for task offloading and resource allocation in a three-tier edge-cloud environment (end devices, edge, and cloud layers) designed to handle

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS AND CORRESPONDING DESCRIPTIONS

Notations	Definition and Description
$\mathcal{N}$	Set of end devices ($\mathcal{N} = \{1, 2, \cdots, N\}$)
$\mathcal{M}$	Set of BSs ($\mathcal{M} = \{1, 2, \cdots M\}$)
$\mathcal{N}_m$	Set of end devices connected to BS m
$C_n^{\mathcal{N}}$	Number of CPU cores at end device n
$C_m^{\mathcal{M}}$	Number of CPU cores over all edge servers at BS m
$C^{\mathcal{C}}$	Number of CPU cores over all cloud servers at cloud layer
K_m	Number of sub-channels of BS m
$\rho_{n,m}^k$	Indicator of whether or not BS m's kth sub-channel is allocated to device n
$\varphi_{n,m}^k$	Signal-to-noise ratio of BS m's kth sub-channel used by device n
$h_{n,m}^k$	Channel gain (dB) between device n and BS m on the kth sub-channel
σ^2	Power of the additive white Gaussian noise (dBm) of a sub-channel
$\eta_{n,m}^k$	The kth sub-channel efficiency between device n and BS m
$\mathcal{B}_m$	Total bandwidth of BS m
b_m^k	Bandwidth assigned to the kth sub-channel at BS m
$\mu_{n,m}^k$	Transmission rate (Gbps) at the kth sub-channel between device n and BS m
$\mu_m^{\mathcal{C}}$	Transmission rate (Gbps) of the wired link from BS m to any cloud server
ψ_n	Task of device n
δ_n	Data size of task ψ_n
ϱ_n	Required computational resources (i.e., CPU cycles) to complete task ψ_n
λ_n	Tolerable latency for task ψ_n
$F_n^{\mathcal{N}}$	Computing capacity available on device n (CPU cycles/sec.)
$f_n^{\mathcal{N}}$	Computing capacity of a single CPU core supported by device n (CPU cycles/sec.)
$F_m^{\mathcal{M}}$	Computing capacity available on edge servers in BS m (CPU cycles/sec.)
$f_m^{\mathcal{M}}$	Computing capacity of a single CPU core supported by an edge server in BS m (CPU cycles/sec.)
$F^{\mathcal{C}}$	Computing capacity available on cloud layer (CPU cycles/sec.)
$f^{\mathcal{C}}$	Computing capacity of a single CPU core supported by a cloud server (CPU cycles/sec.)
$\dot{\mathcal{T}}_n^{\mathcal{N}}$	Delay to execute the task allocated at device n
$\dot{\mathcal{T}}_{n,m}^{\mathcal{M}}$	Delay to execute the allocated task of device n at BS m
$\dot{\mathcal{T}}_n^{\mathcal{C}}$	Delay to execute the allocated task of device n at a cloud server
$\ddot{\mathcal{T}}_{n,m}^{\mathcal{M}}$	Transmission delay for transmitting the allocated task of device n to BS m
$\ddot{\mathcal{T}}_{n,m}^{\mathcal{C}}$	Transmission delay for transmitting the allocated task of device n to a cloud server through BS m
$\dot{\mathcal{E}}_n^{\mathcal{N}}$	Energy to execute the task allocated at device n
$\dot{\mathcal{E}}_{n,m}^{\mathcal{M}}$	Energy to execute the allocated task of device n at BS m
$\dot{\mathcal{E}}_n^{\mathcal{C}}$	Energy to execute the allocated task of device n at a cloud server
$\ddot{\mathcal{E}}_{n,m}^{\mathcal{M}}$	Transmission energy for transmitting the allocated task of device n to BS m
$\ddot{\mathcal{E}}_{n,m}^{\mathcal{C}}$	Transmission energy for transmitting the allocated task of device n to a cloud server through BS m
$\dot{e}_n^{\mathcal{N}}$	Unit energy consumption (J/Gcycle) of execution in device n
$\dot{e}_m^{\mathcal{M}}$	Unit energy consumption (J/Gcycle) of execution in edge m
$\dot{e}^{\mathcal{C}}$	Unit energy consumption (J/Gcycle) of execution in a cloud server
$\ddot{x}_{n,m}^{\mathcal{M}}$	Unit energy consumption (Watt) for transmission from device n to BS m
$\ddot{x}_m^{\mathcal{C}}$	Unit energy consumption (Watt) for wired transmission between BS m and any cloud server

computation-intensive tasks efficiently. The *end devices layer* includes sensors, vehicles, and machines connected to local BSs via wireless networks, generating tasks represented by the set $\mathcal{N}$. The *edge computing layer* consists of M BSs ($\mathcal{M}$), each with edge servers for partial task offloading from devices via wireless links. The *cloud computing layer* comprises high-capacity servers for intensive computation, where tasks are sent to a BS and then forwarded via wired links to the cloud. Cloud server selection depends on the provider's infrastructure and allocation policies.

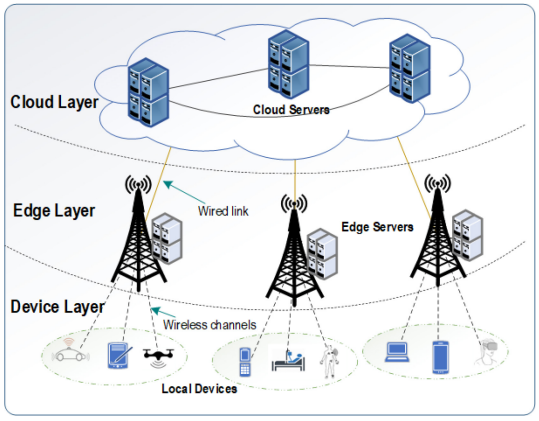


Fig. 1. Three-tier computing architecture comprising end devices, edge, and cloud layers.

B. Task Model

Suppose a device $n \in \mathcal{N}$ is connected to its nearest BS $m \in \mathcal{M}$. In time step t, a device n connected to the nearby BS m randomly generates a task, denoted by $\psi_n = (\delta_n, \varrho_n, \lambda_n)$, where δ_n represents the data size (in MB), ϱ_n denotes the computational resources (CPU cycle) required to complete the task, and λ_n represents the tolerable latency (second) acceptable for this task. The partial offloading decision for the task ψ_n is denoted by the three variables x_n, y_n, and z_n, with values between 0.0 and 1.0, that represent the ratios of tasks computed by the local device, an edge server, and a cloud server, respectively. The partial offloading decision for the task ψ_n can be formulated as follows:

$$x_n + y_n + z_n = 1 \tag{1}$$

C. Communication Model

In this study, we consider a system that uses OFDMA as the multiple access scheme [42]. OFDMA is adopted in each BS to avoid intra-cell interference. The operational frequency band $\mathcal{B}_m$ (MHz) owned by a BS m is divided into K_m orthogonal sub-channels. The end device associated with the closest BS m can utilize an available sub-channel to offload the task to the edge or cloud. The set of devices connected to the BS m is expressed as $\mathcal{N}_m$. The binary variable $\rho_{n,m}^k \in \{0, 1\}$ defines an indicator, where $\rho_{n,m}^k = 1$ indicates that the kth sub-channel in the BS m is allocated to the device n ($1 \leq k \leq K$).

For the kth sub-channel in the BS m, the efficiency of the sub-channel $\eta_{n,m}^k$ can be approximated using Shannon's formula as follows:

$$\eta_{n,m}^k = \log_2\left(1 + \varphi_{n,m}^k\right) \tag{2}$$

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

where $\varphi_{n,m}^{k}$ denotes the signal-to-noise ratio of the kth sub-channel used by device n and it can be expressed as follow:

$$\varphi_{n,m}^{k} = \frac{\ddot{x}_{n,m}^{\mathcal{M}} h_{n,m}^{k}}{\sigma^2} \tag{3}$$

where $\ddot{x}_{n,m}^{\mathcal{M}}$ denotes the transmission power between the device n and the BS m, $h_{n,m}^{k}$ denotes the channel gain (dB) between the device n and the BS m on the kth sub-channel, and σ^2 represents the power of the additive white Gaussian noise of a sub-channel. When the device n is connected to the BS m through the kth sub-channel, the achievable transmission rate $\mu_{n,m}^{k}$ is given by

$$\mu_{n,m}^{k} = b_{m}^{k} \eta_{n,m}^{k} \tag{4}$$

where b_{m}^{k} represents the bandwidth assigned to the kth sub-channel at the BS m.

On the other hand, $\mu_{m}^{\mathcal{C}}$ represents the transmission rate of the wired link from the BS m to any cloud server. The determination of a cloud server is generally based on load balancing techniques employed by the cloud provider; however we do not advocate for a specific method, or cloud server.

D. Delay and Energy Model

In the proposed scheme, the task of device n is divided into three parts for partial execution. The first part, x_n, is executed by the end device itself, the second part, y_n, is executed by the connected BS's edge server, and the third part, z_n, is executed by a cloud server. If the device n can execute the entire task by itself, then the offloading ratios to edge and cloud are zero, as indicated by Eqn. (1) (i.e., $x_n = 1$, $y_n = 0$, and $z_n = 0$). In the following discussion, describe execution at the local, edge, and cloud levels in greater detail.

1) Local Computing: We define $F_{n}^{\mathcal{N}}$ as the computation capacity (CPU cycles/sec.) available on a device n, and $f_{n}^{\mathcal{N}}$ as the computation capacity (or available computational resources) of a single CPU core supported by the device n. Assuming that the device has comprises $C_{n}^{\mathcal{N}}$ CPU cores, its computation capacity can be expressed as $F_{n}^{\mathcal{N}} = C_{n}^{\mathcal{N}} * f_{n}^{\mathcal{N}}$. In this context, a CPU core is allocated to execute a task with a data ratio of x_n on a local device. Then, the delay to execute the task allocated at the device n can be calculated as follows:

$$\dot{\mathcal{T}}_{n}^{\mathcal{N}} = \frac{\varrho_n x_n}{f_{n}^{\mathcal{N}}} \tag{5}$$

Moreover, the energy consumed to execute the task allocated at the device n can be expressed as follows.

$$\dot{\mathcal{E}}_{n}^{\mathcal{N}} = \dot{e}_{n}^{\mathcal{N}} \varrho_n x_n \tag{6}$$

where $\dot{e}_{n}^{\mathcal{N}}$ represents the unit energy consumption (J/Gcycle) of execution in the device n.

2) Edge Computing: To offload task data of ratio y_n to the edge at the BS m, specification of transmission resources such as channel bandwidth, is required. In the proposed communication model, the delay in transmitting the allocated task of the device n to the BS m is given by

$$\ddot{\mathcal{T}}_{n,m}^{\mathcal{M}} = \frac{\delta_n y_n}{\mu_{n,m}^{k}} \tag{7}$$

The energy consumed for transmission of the task data of ratio y_n to the BS m is given by

$$\ddot{\mathcal{E}}_{n,m}^{\mathcal{M}} = \ddot{x}_{n,m}^{\mathcal{M}} \frac{\delta_n y_n}{\mu_{n,m}^{k}} \tag{8}$$

where $\ddot{x}_{n,m}^{\mathcal{M}}$ (Watt) denotes the unit energy consumption for transmission from the device n to the BS m.

We define $F_{m}^{\mathcal{M}}$ as the computing capacity (CPU cycles/sec.) available on edge servers in the BS m, and $f_{m}^{\mathcal{M}}$ as the computation capacity (or available computational resources) of a single CPU core supported by an edge server in the BS m. Assuming that the BS comprises $C_{m}^{\mathcal{M}}$ CPU cores over edge servers, we can express its computation capacity as $F_m = C_{m}^{\mathcal{M}} * f_{m}^{\mathcal{M}}$. In this context, a CPU core is allocated to execute a task with a data ratio of y_n on an edge server. Then, the delay to execute the allocated task of the device n at the BS m can be formulated as follows:

$$\dot{\mathcal{T}}_{n,m}^{\mathcal{M}} = \frac{\varrho_n y_n}{f_{m}^{\mathcal{M}}}, \tag{9}$$

and the energy consumption at the BS m can be calculated as follows:

$$\dot{\mathcal{E}}_{n,m}^{\mathcal{M}} = \dot{e}_{m}^{\mathcal{M}} \varrho_n y_n \tag{10}$$

where $\dot{e}_{m}^{\mathcal{M}}$ denotes the unit energy consumption (J/Gcycle) for execution by an edge server in the BS m.

3) Cloud Computing: The task offloading ratio to a cloud server for the device n is denoted by z_n. The remaining portion of the task with respect to local and edge execution is represented by $z_n = 1 - x_n - y_n$. In our communication model, the delay for transmitting the allocated task of device n to any cloud server via the BS m is given by

$$\ddot{\mathcal{T}}_{n,m}^{\mathcal{C}} = \frac{\delta_n z_n}{\mu_{n,m}^{k}} + \frac{\delta_n z_n}{\mu_{m}^{\mathcal{C}}}. \tag{11}$$

Then, the energy consumed to transmit the task data of ratio z_n to a cloud server via the BS m is given by

$$\ddot{\mathcal{E}}_{n,m}^{\mathcal{C}} = \ddot{x}_{n,m}^{\mathcal{M}} \frac{\delta_n z_n}{\mu_{n,m}^{k}} + \ddot{x}_{m}^{\mathcal{C}} \frac{\delta_n z_n}{\mu_{m}^{\mathcal{C}}} \tag{12}$$

where $\ddot{x}_{m}^{\mathcal{C}}$ (Watt) is denotes the unit energy consumption for wired transmission between the BS m and a cloud server.

We define $F^{\mathcal{C}}$ to be the total computation capacity (CPU cycles/sec.) available on the cloud layer, and $f^{\mathcal{C}}$ to be the computation capacity (or available computational resources) of a single CPU core supported by a cloud server. Assuming that the cloud comprises $C^{\mathcal{C}}$ CPU cores over all cloud servers, we can express the total cloud computation capacity as $F^{\mathcal{C}} = C^{\mathcal{C}} * f^{\mathcal{C}}$. In this context, a CPU core is allocated to execute a task with a data ratio of z_n on a cloud server. Then, the delay to execute the allocated task of the device n on a cloud server can be formulated as follows:

$$\dot{\mathcal{T}}_{n}^{\mathcal{C}} = \frac{\varrho_n z_n}{f^{\mathcal{C}}}. \tag{13}$$

The energy consumption to compute the task allocated on any cloud server can be calculated as follows:

$$\dot{\mathcal{E}}_{n}^{\mathcal{C}} = \dot{e}^{\mathcal{C}} \varrho_n z_n \tag{14}$$

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

where $\dot{e}^{\mathcal{C}}$ denotes the unit energy consumption (J/Gcycle) for execution in a cloud server.

After the partial offloading decision for a task ψ_n is made, which entails determining the values of x_n, y_n, and z_n, the local device starts to execute the partial task data of ratio x_n. Simultaneously, it offloads the partial task data of ratio y_n and z_n to the edge and cloud servers, respectively. As a result, the overall delay $\mathcal{T}_n$ for a partial task offloading can be computed as follows:

$$\mathcal{T}_n(x_n, y_n, z_n) = \max\left\{\dot{\mathcal{T}}_n^{\mathcal{N}},\ \ddot{\mathcal{T}}_{n,m}^{\mathcal{M}} + \dot{\mathcal{T}}_{n,m}^{\mathcal{M}},\ \ddot{\mathcal{T}}_{n,m}^{\mathcal{C}} + \dot{\mathcal{T}}_n^{\mathcal{C}}\right\} \quad (15)$$

It should be noted that the overall delay is determined by the maximum delay among the local device, edge server, and cloud server during task offloading and execution. We can also define the indicator function to determine whether the acceptable latency of each task is satisfied:

$$\mathcal{I}_n(x_n, y_n, z_n) = \begin{cases} 1 \text{ if } \mathcal{T}_n(x_n, y_n, z_n) \le \lambda_n, \\ 0 \text{ otherwise.} \end{cases} \quad (16)$$

On the other hand, the energy consumption $\mathcal{E}_n$, can be computed as follows:

$$\mathcal{E}_n(x_n, y_n, z_n) = \dot{\mathcal{E}}_n^{\mathcal{N}} + \left(\ddot{\mathcal{E}}_{n,m}^{\mathcal{M}} + \dot{\mathcal{E}}_{n,m}^{\mathcal{M}}\right) + \left(\ddot{\mathcal{E}}_{n,m}^{\mathcal{C}} + \dot{\mathcal{E}}_n^{\mathcal{C}}\right). \quad (17)$$

Energy consumption of a task equals the total energy used by the device, edge server, and cloud server during offloading and execution.

IV. Problem Formulation and Hierarchical Markov Decision Process

A. Problem Formulation

Our objective is to minimize total energy consumption while maximizing the number of tasks meeting latency constraints.

For all $n \in \mathcal{N}$, find x_n, y_n, z_n to:

$$P1: \quad \min \sum_{n\in\mathcal{N}} \mathcal{E}_n(x_n, y_n, z_n) \quad (18)$$

$$P2: \quad \max \sum_{n\in\mathcal{N}} \mathcal{I}_n(x_n, y_n, z_n) \quad (19)$$

subject to

$$\begin{aligned}
&C1: && x_n, y_n, z_n \in [0, 1], \quad \forall n \in \mathcal{N} \\
&C2: && x_n + y_n + z_n = 1, \quad \forall n \in \mathcal{N} \\
&C3: && \sum_{n\in\mathcal{N}_m} \rho_{n,m}^k \le 1, \quad \forall m \in \mathcal{M}, \forall k \in \{1, \ldots, K_m\} \\
&C4: && \sum_{k=1}^{K_m} \rho_{n,m}^k \le 1, \quad \forall m \in \mathcal{M}, \forall n \in \mathcal{N}_m \\
&C5: && \sum_{n\in\mathcal{N}_m} \sum_{k=1}^{K_m} b_m^k \rho_{n,m}^k \le \mathcal{B}_m, \quad \forall m \in \mathcal{M} \\
&C6: && f_n^{\mathcal{N}} \le F_n^{\mathcal{N}}, \quad \forall n \in \mathcal{N} \\
&C7: && \sum_{n\in\mathcal{N}_m} f_m^{\mathcal{M}} \le F_m^{\mathcal{M}}, \quad \forall m \in \mathcal{M} \\
&C8: && \sum_{n\in\mathcal{N}} f^{\mathcal{C}} \le F^{\mathcal{C}}
\end{aligned} \quad (20)$$

Constraints $C1$ and $C2$ define task offloading ratio ranges and ensure they sum to one; $C3$ and $C4$ restrict sub-channel allocation so each sub-channel serves at most one device and vice versa [43]. $C5$ limits total bandwidth per BS. $C6$, $C7$, and $C8$ restrict computational resource allocations at local devices, edge servers, and cloud servers, respectively.

The joint optimization problem ($P1$, $P2$) is a complex multi-objective problem with continuous, coupled decision variables. Load levels at edge and cloud servers depend on task parameters (data size, CPU cycles, latency), task arrival times, and other devices' decisions. Given unknown future tasks and dynamic resource availability, we propose an HDRL-PTO scheme tailored for dynamic network environments.

B. Hierarchical Markov Decision Process

The proposed hierarchical approach splits the problem's MDP into two: a high-level MDP (HL MDP) and a low-level MDP (LL MDP). Their designs are detailed below.

1) HL MDP: The HL agent observes global network states, including device tasks and available resources, and makes partial offloading decisions using a discrete action strategy. The HL MDP model can be described by a tuple: $\langle \mathcal{S}^h, \mathcal{U}^h, \mathcal{F}^h, R^h, \gamma^h \rangle$. At every time step t within the HL horizon, $s_t^h \in \mathcal{S}^h$ corresponds to a state received by HL agent receives from the given network environment, The HL agent takes an action $u_t^h \in \mathcal{U}^h$. $\mathcal{F}^h(s_{t+1}^h | s_t^h, u_t^h)$: $\mathcal{S}^h \times U^h \to \mathcal{S}^h$ is the state transition function, which describes the evolution of HL states based on the current HL state and action. However, the HL agent does not have complete knowledge of this function. The reward function of the HL agent is given by $r_t^h = R^h(s_t^h, u_t^h)$: $\mathcal{S}^h \times \mathcal{U}^h \to \mathcal{R}$. The reward discount is defined to be $\gamma^h \in [0, 1)$. The state space, action space, and reward function used in the HL MDP model are defined as follows.

a) HL state: At every time step t, the state $s_t^h \in \mathcal{S}^h$ consists of six components $\{T, \mathcal{R}, n_t, \mathcal{F}_t^{\mathcal{L}}, \mathcal{F}_t^{\mathcal{E}}, F_t^{\mathcal{C}}\}$, where T and $\mathcal{R}$ remain constant throughout all time steps within an episode. These components are described below:

- $T = (\bar{\psi}_1, \ldots, \bar{\psi}_{|\mathcal{N}|})$: The set of tasks generated on each device. For each device n, $\bar{\psi}_n$ is allocated to $\psi_n (= (\delta_n, \varrho_n, \lambda_n))$ if there is a task on the device ($1 \le n \le |\mathcal{N}|$). Otherwise, it is allocated to $(0.0, 0.0, 0.0)$.
- $\mathcal{R}$: The 3D matrix where each element $\rho_{n,m}^k \in \{0, 1\}$ is a binary variable, where $\rho_{n,m}^k = 1$ indicates that the kth sub-channel in the BS m is allocated to the device n ($1 \le k \le K_m$, $1 \le m \le |\mathcal{M}|$, and $1 \le n \le |\mathcal{N}|$).
- n_t: The index of the device from which an action needs to be extracted at the time step t. It is selected sequentially in $\mathcal{N}$.
- $\mathcal{F}_t^{\mathcal{L}} = (F_{1,t}^{\mathcal{L}}, \ldots, F_{|\mathcal{N}|,t}^{\mathcal{L}})$: The set of computation capacities available on each device in $\mathcal{N}$, where $F_{n,t}^{\mathcal{L}}$ denotes the computation capacities available on the device n at time step t ($1 \le n \le |\mathcal{N}|$).
- $\mathcal{F}_t^{\mathcal{E}} = (F_{1,t}^{\mathcal{E}}, \ldots, F_{|\mathcal{M}|,t}^{\mathcal{E}})$: The set of computation capacities available on edge servers in each BS in $\mathcal{M}$, where $F_{m,t}^{\mathcal{E}}$ denotes the computation capacities available on the BS m at time step t ($1 \le m \le |\mathcal{M}|$).
- $F_t^{\mathcal{C}}$: The computing capacity available on the cloud layer.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

b) HL action: In the HL action space $\mathcal{U}^h$, the HL agent takes an action u_t^h to decide suitable execution levels (locations) for the task of the device n_t, based on the current observation s_t^h. The sequence of an action, u_t^h, is given by $\langle a, b, c\rangle$, where $a, b, c \in \{0, 1\}$. The following seven actions are considered:

- $\langle 1, 0, 0\rangle$: the task is allocated to the local device
- $\langle 0, 1, 0\rangle$: the task is offloaded to the edge layer
- $\langle 0, 0, 1\rangle$: the task is offloaded to the cloud layer
- $\langle 1, 1, 0\rangle$: the task is divided among the local devices and the edge layer
- $\langle 1, 0, 1\rangle$: the task is divided among into the local device and the cloud layer
- $\langle 0, 1, 1\rangle$: the task is divided among the edge layer and the cloud layer
- $\langle 1, 1, 1\rangle$: the task is divided among the local device, the edge layer, and the cloud layer

The aforementioned HL agent decisions are categorized into three types:

- *Type I* ($\langle 1, 0, 0\rangle$, $\langle 0, 1, 0\rangle$, and $\langle 0, 0, 1\rangle$): the HL agent chooses to allocate or offload the entire task to a single local, edge, and cloud layer, without any form of task partitioning.
- *Type II* ($\langle 1, 1, 0\rangle$, $\langle 1, 0, 1\rangle$, and $\langle 0, 1, 1\rangle$): the HL agent decides to offload the task to two layers.
- *Type III* ($\langle 1, 1, 1\rangle$): the HL agent decides to offload the task to three layers.

The HL agent uses action masks to eliminate invalid actions, such as offloading to unavailable edge resources (e.g., $\langle 0, 1, 0\rangle$, $\langle 1, 1, 0\rangle$), reducing the action space and speeding up learning. For HL actions of Type II or III, the decision is passed to the LL agent, which determines the task division ratios $(x_{n_t}, y_{n_t}, z_{n_t})$ for the selected locations. The LL MDP design is detailed later.

c) HL reward function: To minimize energy consumption when offloading tasks, while simultaneously maximizing the number of tasks specifying latency constraints. In this context, at the current time step t and for task assigned to device n_t, the reward function can be defined as follows:

$$r_t^h = -c \cdot e_t^h + d \cdot i_t^h \tag{21}$$

where e_t^h represents the normalized energy consumption related to the task assigned to device n_t with a range of 0.0 of 1.0. On the other hand, i_t^h serves as a binary indicator that determines whether the latency constraint is satisfied for the given task of the device n_t, which is defined as follows:

$$i_t^h = \begin{cases} 1, \text{ if } \left(\lambda_{n_t} - \mathcal{T}_{n_t}(x_{n_t}, y_{n_t}, z_{n_t})\right) \geq 0 \\ 0, \text{ otherwise} \end{cases} \tag{22}$$

In Eqn. (23), the specific values of the scaling factors c and d can be determined based on the desired trade-off between the energy consumption and the satisfaction of latency constraint, considering the relative importance of each objective.

d) HL reward function: To minimize energy consumption and maximize tasks meeting latency constraints, the HL reward at time t for device n_t is:

$$r_t^h = -c \cdot e_t^h + d \cdot i_t^h \tag{23}$$

where e_t^h is the normalized energy consumption (range 0–1), and i_t^h is a binary indicator of latency satisfaction:

$$i_t^h = \begin{cases} 1, \text{ if } \left(\lambda_{n_t} - \mathcal{T}_{n_t}(x_{n_t}, y_{n_t}, z_{n_t})\right) \geq 0 \\ 0, \text{ otherwise} \end{cases} \tag{24}$$

Scaling factors c and d balance the trade-off between energy consumption and latency satisfaction based on their relative importance.

2) LL MDP: After receiving the action from the HL agent (Type II or III), the LL agent selects the optimal offloading ratio for the end device, edge layer, and/or cloud layer. If the HL action is Type II, the LL agent completes its action in a single step within the LL horizon; if it is Type III, the LL agent completes its action in two steps within the LL horizon. he LL MDP model can be defined as a tuple $\langle \mathcal{S}^l, \mathcal{U}^l, \mathcal{F}^l, R^l, \gamma^l\rangle$. At the two time steps, $k = 0$ or 1, within the LL horizon, $s_k^l \in \mathcal{S}^l$ corresponds to the state received by the LL agent from the given network environment, The LL agent takes an action $u_k^l \in \mathcal{U}^l$. $\mathcal{F}^l(s_{k+1}^l | s_k^l, u_k^l) \colon \mathcal{S}^l \times U^l \to \mathcal{S}^l$ denotes the state transition function, which describes the evolution of LL states evolve based on the current LL state and action. The reward function of the LL agent is given by $r_k^l = R^l(s_k^l, u_k^l) \colon \mathcal{S}^l \times \mathcal{U}^l \to \mathcal{R}$. The reward discount is defined to be $\gamma^l \in [0, 1)$.

a) LL state: In the LL state space $\mathcal{S}^l$, the LL agent receives a state s_k^l, where $k = 0$ or 1 in the LL horizon. In the first time step ($k = 0$), the state $s^l(0) \in \mathcal{S}^l$ consists of seven components $\{T, \mathcal{R}, n_t, \mathcal{F}_t^{\mathcal{L}}, \mathcal{F}_t^{\mathcal{E}}, F_t^{\mathcal{C}}, u_t^h\}$. Among these components, the first six correspond to the same values as the first six in the HL state, and the final component represents the HL action, u_t^h, taken at time step t within the HL horizon. At the next time step ($k = 1$), the state $s^l(1) \in \mathcal{S}^l$ also consists of seven components. Among them, $\mathcal{F}_t^{\mathcal{L}}$, $\mathcal{F}_t^{\mathcal{E}}$, and $F_t^{\mathcal{C}}$ are updated in accordance with the decision made by the LL agent.

b) LL action: In the LL action space $\mathcal{U}^l$, the LL agent takes an action u_k^l, where $k = 0$ or 1. The LL agent decides to divide the task in a particular ratio and the LL action value always lies between 0.0 and 1.0. If u_t^h is of *Type II*, the LL agent decides on a single-step action, $u^l(0)$, referring to the task allocation or offloading ratio between the two layers specified by u_t^h. If u_t^h is of *Type III*, the LL agent makes a two-step decision with two actions, $u^l(0)$ and $u^l(1)$, that determine the task allocation or offloading ratio between the three layers specified by u_t^h.

c) LL reward function: The LL agent aims to optimally divide the offloaded task by selecting appropriate ratios to optimize the objectives. Its reward matches the HL agent's since the HL action depends on the LL decision. The difference is that the LL agent focuses on individual tasks, while the HL agent considers all tasks collectively. The LL agent completes its action in one or two steps and receives the reward after the final step. When the last step is $\bar{k}$ within the LL horizon, the reward function of the LL agent is defined as follows:

$$r_k^l = \begin{cases} 0.0 & \text{if } k \neq \bar{k} \\ -c \cdot e_k^l + d \cdot i_k^l & \text{if } k = \bar{k} \end{cases} \tag{25}$$

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

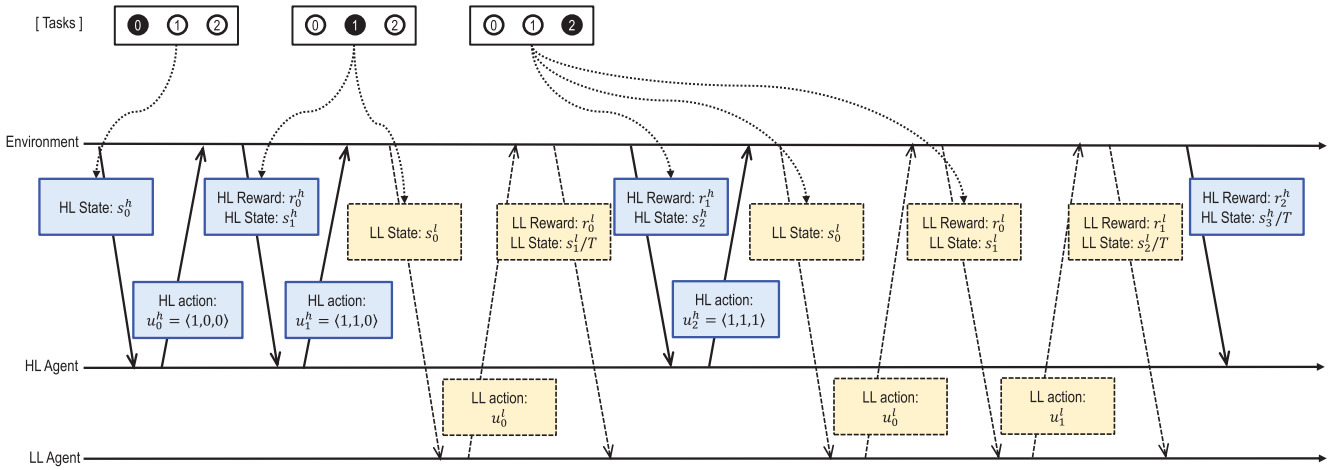


Fig. 2. The information flow at the proposed hierarchical MDP framework ($*/T$ refers to the terminal state at HL or LL).

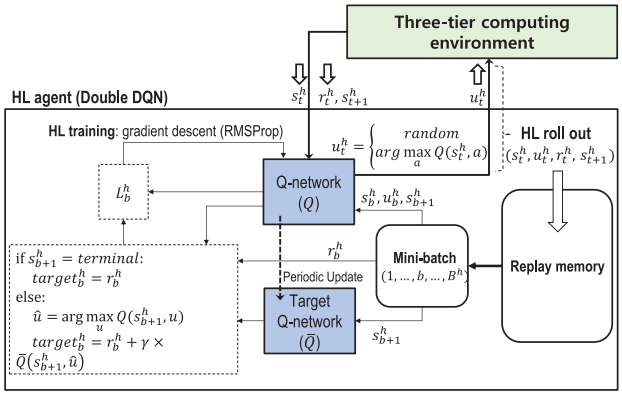


Fig. 3. Double DQN learning process of HL agent.

where $0 \le k \le 1$, $0 \le \bar{k} \le 1$, and e_k^l and i_k^l are identical to the corresponding values in the HL reward, i.e., $e_k^l = e_t^h$ and $i_k^l = i_t^h$.

The information flow in the proposed hierarchical MDP framework is illustrated in Fig. 2.

V. Policy Learning for Partial Task Offloading

This section presents learning and operation algorithms based on the proposed hierarchical MDP for partial task offloading and resource allocation in a three-tier computing architecture.

A. HL Policy Learning by Double DQN

Fig. 3 illustrates the HL agent's Double DQN learning process. The agent observes state s_t^h, selects action u_t^h using an epsilon-greedy policy, stores the transition $(s_t^h, u_t^h, r_t^h, s_{t+1}^h)$ in replay memory, and updates the Q-network from these experiences. The exploration rate ϵ^h decays over time to balance exploration and exploitation. During training, a mini-batch is extracted from the replay memory. The TD-error for a mini-batch transition $(s_b^h, u_b^h, r_b^h, s_{b+1}^h)$ is computed by comparing the predicted Q-value with the target Q-value based on the reward and the maximum next-state Q-value. We denote the TD-error of the transition by L_b^h which is formally calculated as follows:

$$L_b^h = \left(target_b^h - Q(s_b^h, u_b^h)\right)^2 \tag{26}$$

where

$$target_b^h = \begin{cases} r_b^h & \text{if } s_{b+1}^h = \text{terminal}, \\ r_b^h + \gamma^h \bar{Q}\left(s_{b+1}^h, \text{argmax}_u Q(s_{b+1}^h, u)\right) & \text{otherwise}, \end{cases} \tag{27}$$

Here, Q and $\bar{Q}$ denote the Q-network and target Q-network, respectively. Both share the same structure, but $\bar{Q}$'s weights are periodically updated by copying from Q every fixed number of steps.

The TD-error serves as a measure of the difference between the predicted and target values and is used to update the Q-network parameters via a gradient descent method (e.g., RMSProp [44]) to enhance Q-value estimation. Let θ_q^h denotes the parameter of the Q-network.

$$\theta_q^h = \theta_q^h - \alpha_q^h \frac{\partial}{\partial \theta_q^h} \sum_{b=1}^{B^h} L_b^h \tag{28}$$

where α is the HL training learning rate, B^h is the mini-batch size, and the target Q-network parameters are updated from the Q-network every P^h episodes.

B. LL Policy Learning by PPO

The PPo-based learning process of the LL agent is illustrated in Fig. 4. The actor operates by collecting trajectories from the environment iteratively. For a transition $(s_b^l, u_b^l, r_b^l, s_{b+1}^l)$ in the collected trajectories, the critic estimates the advantage function as follows.

$$A_b = R\left(s_b^l, u_b^l\right) - V\left(s_b^l\right) \tag{29}$$

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

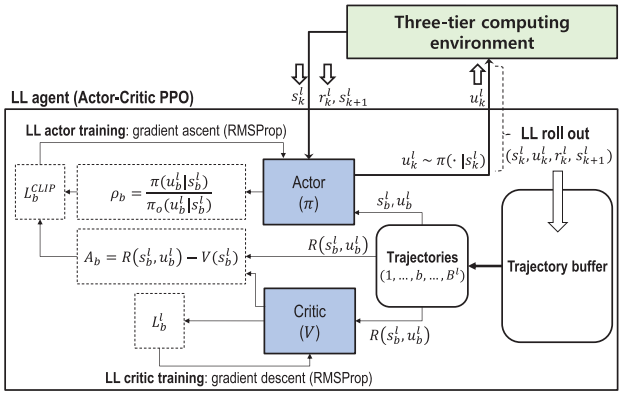


Fig. 4. PPO learning process of LL agent.

where $R(s_b^l, u_b^l)$ corresponds to the reward-to-go obtained from the specific state-action pair:

$$R\left(s_b^l, u_b^l\right) = \sum_{b}^{T_b - 1} r_b^l, \tag{30}$$

where T_b refers to the terminal state's time step, and $V(s_b^l)$ denotes the value estimated for the state, representing the expected return.

The advantage function helps the LL agent determine the quality of its LL actions corresponding to different states. PPO uses the following surrogate objective to update the actor's LL policy.

$$L_b^{CLIP} = min\left(\rho_b \cdot A_b, clip\left(\rho_b, 1 - \eta^l, 1 + \eta^l\right) \cdot A_b\right) \tag{31}$$

where η^l denotes to the clip parameter controlling the range of policy updates and ρ_b denotes the ratio of the new policy π to the old policy π_o, as follows:

$$\rho_b = \frac{\pi\left(u_b^l | s_b^l\right)}{\pi_o\left(u_b^l | s_b^l\right)}. \tag{32}$$

The objective guides the LL policy toward actions with higher advantage estimates, maximizing expected return while ensuring updates remain close to the previous policy to prevent destabilizing large changes. Let θ_π^l denote the LL policy parameters; the surrogate objective is optimized via stochastic gradient ascent as

$$\theta_\pi^l = \theta_\pi^l + \alpha_\pi^l \frac{\partial}{\partial \theta_\pi^l} \sum_{b=1}^{B^l} L_b^{CLIP}, \tag{33}$$

where α_π^l is the LL policy learning rate and B^l is the trajectory batch size. The critic's value function is updated by minimizing the mean-squared error between observed returns and estimated values:

$$L_b^l = \left(R\left(s_b^l, u_b^l\right) - V\left(s_b^l\right)\right)^2, \tag{34}$$

and with θ_v^l as the critic's parameters, the update follows

$$\theta_v^l = \theta_v^l - \alpha_v^l \frac{\partial}{\partial \theta_v^l} \sum_{b=1}^{B^l} L_b^l, \tag{35}$$

where α_v^l is the critic's learning rate.

Algorithm 1: Learning of HL and LL Policies

Initialize Devices, tasks, and resources at three layers
Initialize HL Q-network Q with random weights θ_q^h
Initialize HL target Q-network $\bar{Q}$ with weights $\bar{\theta}_q^h$
Initialize $\bar{\theta}_q^h \leftarrow \theta_q^h$
Initialize LL policy network π with weights θ_π^l
Initialize LL value network V with weights θ_v^l
Initialize Replay memory R for HL agent
Initialize Trajectory buffer J for LL agent
Initialize γ^h, P^h, α_q^h, γ^l, η^l, α_π^l, and α_v^l
for *episode* $= 1, 2, 3, \ldots, E$ **do**
  $t \leftarrow 0$
  reset environment and get the HL state $s_t^h (= s_0^h)$
  while $t < \mathcal{N}$ **do**
    if *a task exists in the device* $n_t \in s_t^h$ **then**
      /* HL roll out - Part I */
      if *a random value* $< \epsilon^h$ **then**
        select a random HL action u_t^h
      else
        select an HL action $u_t^h = \text{argmax}_u Q(s_t^h, u)$
      take the HL action u_t^h into environment
      if u_t^h *falls into Type II or III* **then**
        /* LL roll out */
        $k \leftarrow 0$
        get the LL state $s_k^l (= s_0^l)$
        while $k \leq 1$ **do**
          sample an LL action $u_k^l \sim \pi(\cdot \mid s_k^l)$
          take the LL action u_k^l into environment
          get the LL reward r_k^l and the next LL state s_{next}^l
          store the transition $(s_k^l, u_k^l, r_k^l, s_{next}^l)$ into J
          if $u_t^h \in$ *Type II* **then**
            break
          else
            $k \leftarrow k + 1$
            $s_k^l \leftarrow s_{next}^l$
        /* LL optimization */
        if $|J| >= B^l$ **then**
          update π with Eqn. (29), (30), (31), (32), and (33)
          update V with Eqn. (30), (34) and (35)
          make J empty
      /* HL roll out - Part II */
      get the HL reward r_t^h and the next HL state s_{next}^h
      store the transition $(s_t^h, u_t^h, r_t^h, s_{next}^h)$ into R
      /* HL optimization */
      if $|R| >= B^h$ **then**
        update Q with Eqn. (26), (27), and (28)
    else
      $s_{next}^h = s_t^h$
    $t \leftarrow t + 1$
    $s_t^h = s_{next}^h$
  Every P^h episodes, $\bar{\theta}_q^h \leftarrow \theta_q^h$
$\widetilde{\theta}_q^h \leftarrow \theta_q^h$ and $\widetilde{\theta}_\pi^l \leftarrow \theta_\pi^l$
Return $\widetilde{\theta}_q^h$ and $\widetilde{\theta}_\pi^l$

Algorithm 1 presents the overall learning process for HL and LL policies. It initializes the network configurations, neural networks, and hyperparameters. The algorithm alternates between HL and LL rollouts, optimizing the HL Q-network (Q), LL policy network (π), and LL value network (V). The HL policy selects actions using Q, while the LL actor and critic use π and V to sample actions and estimate state values.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Operation With Trained HL and LL Policies

Initialize Devices, tasks, and resources at three layers
Initialize HL Q-network Q with the trained weights $\widetilde{\theta}_q^h$
Initialize LL policy network π with the trained weights $\widetilde{\theta}_\pi^l$
$t \leftarrow 0$
reset environment and obtain the HL state $s_t^h (= s_0^h)$
while $t < \mathcal{N}$ **do**
  if *a task exists in the device* $n_t \in s_t^h$ **then**
    /* HL roll out - Part I */
    select an HL action $u_t^h = \text{argmax}_u Q(s_t^h, u)$
    take the HL action u_t^h into environment
    if u_t^h *falls into Type II or III* **then**
      /* LL roll out */
      $k \leftarrow 0$
      get the LL state $s_k^l (= s_0^l)$
      while $k \leq 1$ **do**
        sample an LL action $u_k^l \sim \pi(\cdot \mid s_k^l)$
        take the LL action u_k^l into environment
        obtain the LL reward r_k^l and the following LL state s_{next}^l
        if $u_t^h \in$ *Type II* **then**
          break
        else
          $k \leftarrow k + 1$
          $s_k^l \leftarrow s_{next}^l$
    /* HL roll out - Part II */
    obtain the HL reward r_t^h and the next HL state s_{next}^h
  else
    $s_{next}^h = s_t^h$
  $t \leftarrow t + 1$
  $s_t^h = s_{next}^h$

Transitions are stored in replay memory and trajectory buffers, with network parameters updated via Eqns. (26) to (35). The target Q-network is updated every P^h episodes. This process repeats until reaching the maximum episodes (E) and time steps ($\mathcal{N}$), improving policies and value estimates. The final trained parameters $\widetilde{\theta}_q^h$ and $\widetilde{\theta}_\pi^l$ are obtained for the HL Q-network and LL policy network, respectively. Algorithm 2 describes the operation of the trained HL and LL policies. It starts by initializing the network configuration and loading the HL Q-network and LL policy network with trained parameters $\widetilde{\theta}_q^h$ and $\widetilde{\theta}_\pi^l$. The inference process then selects HL actions based on Q-values and samples LL actions from the LL policy while interacting with the environment and collecting rewards. This continues until the maximum time steps, $\mathcal{N}$, are reached, enabling decision-making via the learned policies.

VI. Performance Evaluation

A. Simulation Environment Setup

The simulation environment is configured using i9-10900K CPU with a 64GB RAM and RTX 3090 GPU with Linux Ubuntu 20.04.02 LTS with Python 3.8 and PyTorch 1.9.0. The performances of the proposed HDRL-PTO algorithm and two other DRL algorithms are compared: 1) 'Deep Reinforcement Learning for Partial Task Optimization (DRL-PTO)' and 2) 'Deep Reinforcement Learning for Task Optimization (DRL-TO)'. Like HDRL-PTO, DRL-PTO adopts a partial task offloading strategy in a three-tier architecture. However, DRL-PTO utilizes a non-hierarchical DRL approach, specifically employing the PPO algorithm. In DRL-PTO, the agent always determines the values of the three variables x_n, y_n, and z_n, without relying on a discrete decision process for offloading computation levels. Contrary to partial task offloading strategies, DRL-TO adopts a non-partial task offloading approach, where the entire task is either executed locally on the device or offloaded to an edge server or a cloud server. These schemes are tested in two types of environments, Network I and Network II. The configuration details for both network environments are listed in Tables II and III. The distinction between Networks I and II, in terms of their varying levels of resource availability, highlights the implications for their respective capabilities in wireless communication and computing operations. During training and testing, task data size, computational resources, and latency tolerance are uniformly sampled within the range in Table IV. Double DQN and PPO parameters are listed in Table V. Reward scaling parameters c and d in (23) and (25) are both 0.5. The maximum training episodes E is 120, 000.

TABLE II
Configuration Parameters of **Network I** and **II**

	Network I	**Network II**
N	96	192
$\mathcal{N}$	$\{1, \cdots, 96\}$	$\{1, \cdots, 192\}$
M	3	3
$\mathcal{M}$	$\{1, 2, 3\}$	$\{1, 2, 3\}$
$\mathcal{N}_m$	$\mathcal{N}_1 = \{1, \cdots, 32\}$ $\mathcal{N}_2 = \{33, \cdots, 64\}$ $\mathcal{N}_3 = \{65, \cdots, 96\}$	$\mathcal{N}_1 = \{1, \cdots, 64\}$ $\mathcal{N}_2 = \{65, \cdots, 128\}$ $\mathcal{N}_3 = \{129, \cdots, 192\}$
$C_n^{\mathcal{N}}$	$C_1^{\mathcal{N}} = \cdots = C_{96}^{\mathcal{N}} = 1$	$C_1^{\mathcal{N}} = \cdots = C_{192}^{\mathcal{N}} = 1$
$C_m^{\mathcal{M}}$	$C_1^{\mathcal{M}} = C_2^{\mathcal{M}} = C_3^{\mathcal{M}} = 32$	$C_1^{\mathcal{M}} = C_2^{\mathcal{M}} = C_3^{\mathcal{M}} = 64$
$C^{\mathcal{C}}$	48	144
K_m	$K_1 = K_2 = K_3 = 32$	$K_1 = K_2 = K_3 = 64$
$\rho_{n,m}^k$	$\begin{cases} \rho_{n,m}^k = 1 & \text{if } n = k, \\ \rho_{n,m}^k = 0 & \text{otherwise} \end{cases}$	$\begin{cases} \rho_{n,m}^k = 1 & \text{if } n = k, \\ \rho_{n,m}^k = 0 & \text{otherwise} \end{cases}$
$F_n^{\mathcal{N}}$	1.0 (CPU cycles/sec.)	1.0 (CPU cycles/sec.)
$f_n^{\mathcal{N}}$	1.0 (CPU cycles/sec.)	1.0 (CPU cycles/sec.)
$F_m^{\mathcal{M}}$	$96.0 (= 3.0 * 32)$ (CPU cycles/sec.)	$192.0 (= 3.0 * 64)$ (CPU cycles/sec.)
$f_m^{\mathcal{M}}$	3.0 (CPU cycles/sec.)	3.0 (CPU cycles/sec.)
$F^{\mathcal{C}}$	$240.0 (= 5.0 * 48)$ (CPU cycles/sec.)	$720.0 (= 5.0 * 144)$ (CPU cycles/sec.)
$f^{\mathcal{C}}$	5.0 (CPU cycles/sec.)	5.0 (CPU cycles/sec.)
b_m^k	9.0 (MHz)	10.0 (MHz)
$\mathcal{B}_m$	$288.0 (= 9.0 * 32)$ (MHz)	$640.0 (= 10.0 * 64)$ (Mhz)

B. Result Analysis

The training episode rewards of HDRL-PTO, DRL-PTO, and DRL-TO are shown in Fig. 5 (Network I) and Fig. 6 (Network II). HDRL-PTO consistently achieves higher rewards due to its hierarchical design, which improves learning efficiency, load balancing, and task distribution. In HDRL-PTO, the HL policy selects execution levels, and the LL

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMMON CONFIGURATION PARAMETERS OF **NETWORK I** AND **II**

$\dot{e}_n^{\mathcal{N}}$	1.0 (J/Gcycle)
$\dot{e}_m^{\mathcal{M}}$	0.1 (J/Gcycle)
$\dot{e}^{\mathcal{C}}$	0.003 (J/Gcycle)
$\ddot{x}_{n,m}^{\mathcal{M}}$	40 (Watt)
$\ddot{x}_m^{\mathcal{C}}$	80 (Watt)
$\varphi_{n,m}^k$	$1.27 \times 10^{22} \left(\simeq \frac{40 \times 10^{\frac{0.1}{10}}}{10^{\frac{-174.92-30}{10}}} \right)$ (Eqn. (3))
$h_{n,m}^k$	0.1 (dB)
σ^2	-174.92 (dBm)
$\eta_{n,m}^k$	$7.79 (\simeq \log_2(1 + 10 \cdot log_{10}(1.27 \times 10^{22})))$ (Eqn. (2))
$\mu_{n,m}^k$	$70.11 (\simeq 9.0 * 7.79)$ (Mbps) for Network I (Eqn. (4))
	$77.90 (\simeq 10.0 * 7.79)$ (Mbps) for Network II (Eqn. (4))
$\mu_m^{\mathcal{C}}$	100 (Mbps)

TABLE IV
COMMON TASK PARAMETERS OF **NETWORK I** AND **II**
(THE VALUE IS SAMPLED BASED ON A UNIFORM DISTRIBUTION)

δ_n	$100 \sim 500$ (KByte)
ϱ_n	$500 \sim 1000$ (Bit/cycle)
λ_n	$0.36 \sim 2.0$ (Sec.)

TABLE V
SIMULATION HYPERPARAMETERS AND LEARNING PARAMETERS OF DOUBLE DQN AND PPO

Parameter	**Value**
Number of Q-network hidden layers	2
Number of neurons per hidden layer	256
Capacity of replay memory R	750,000
Exploration rate (ϵ^h,)	0.95 & 0.01
Epsilon decay strategy	Linear
Discount factor (γ^h)	0.99
Learning rate (α_q^h)	1e-5
Batch size (B^h)	256
Target network update frequency	1,000 steps
Optimizer	Adam
Maximum timesteps per episode	500
Number of PPO update iterations	10
Coefficient of value function loss	0.5
Coefficient of entropy regularizer	0.01
PPO train horizon	4,096
PPO clip parameter (η^l)	0.3
Discount factor (γ^l)	0.99
Policy learning rate (α_π^l)	5e-5
Value learning rate (α_v^l)	5e-6
Batch size (B^l)	256
Maximum timesteps per episode	500

policy determines offloading ratios (x_n, y_n, z_n) in a three-tier network, enabling efficient resource utilization. To further validate HDRL-PTO, we compare task acceptance ratios during training, shown in Fig. 7 for Network I and Fig. 8 for Network II. HDRL-PTO outperforms DRL-PTO and DRL-TO, achieving acceptance ratios of 80–90%, compared to 55–65% for DRL-PTO and 40–45% for DRL-TO in both networks. This demonstrates HDRL-PTO's superior ability to maximize tasks meeting latency constraints (Eqns. (23) and (24)) by making optimal offloading decisions that reduce execution delays. In contrast, DRL-PTO and DRL-TO underutilize resources due to lack of task partitioning and inefficient resource allocation, leading to lower performance.

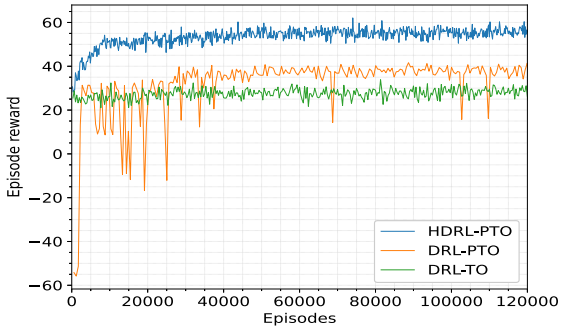


Fig. 5. Network I: Episode rewards of three schemes during training (sum of time-step rewards per episode).

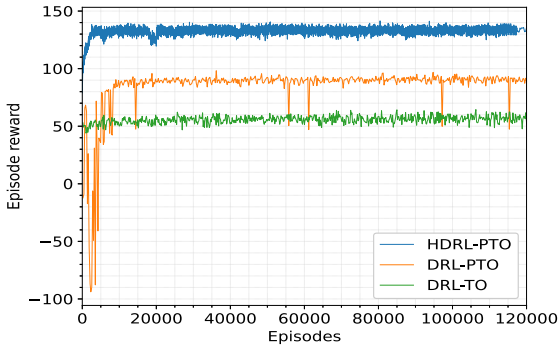


Fig. 6. Network II: Episode rewards of three schemes during training (sum of time-step rewards per episode).

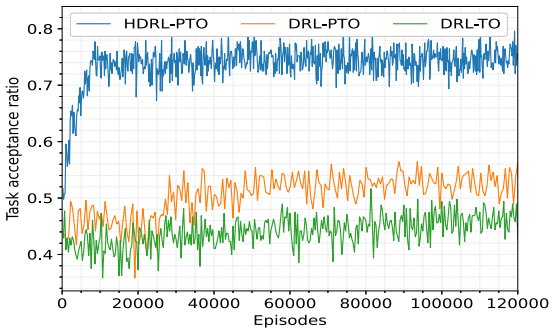


Fig. 7. Network I: Task acceptance ratios of the three schemes during training (sum of indicator functions from all time steps per episode).

Figs. 9 and 10 compare energy consumption of the three schemes. HDRL-PTO minimizes energy while maximizing task acceptance by selecting optimal offloading locations using both HL and LL agents. The HL agent dynamically chooses among *Type I*, *II*, and *III* actions based on energy, latency, and resource availability. In contrast, DRL-PTO relies only on *Type III* actions ($\langle 1, 1, 1 \rangle$), resulting in higher energy use due to always splitting tasks across local, edge, and cloud. DRL-TO uses only *Type I* actions, offloading entire tasks to one location, which limits resource utilization and acceptance ratios despite lower energy use. DRL-PTO's fixed partial offloading to all three layers restricts learning and hinders improvements in acceptance ratio and energy efficiency.

Comparison of HDRL-PTO with DRL-TO in Figs. 7, 8, 9, and 10 highlights the importance of task partitioning to improve task acceptance ratio. Similarly, comparison with

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

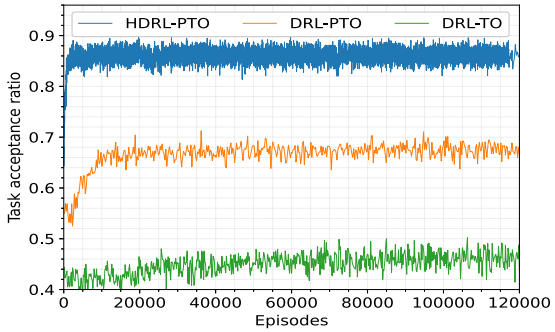


Fig. 8. Network II: Task acceptance ratios of the three schemes during training (sum of indicator functions from all time steps per episode).

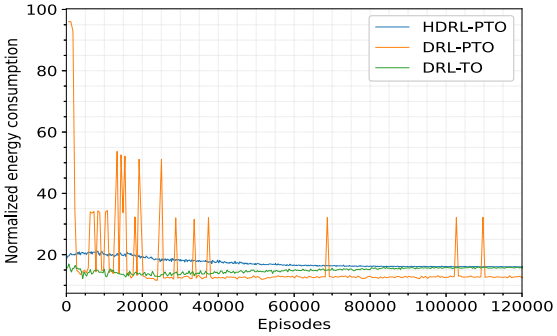


Fig. 9. Network I: Energy consumption of three schemes during training (normalized per episode).

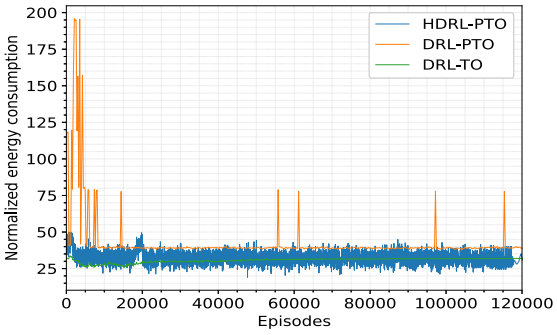


Fig. 10. Network II: Energy consumption of three schemes during training (normalized per episode).

DRL-PTO shows the value of selecting suitable offloading locations and ratios to reduce energy consumption. Both DRL-PTO and DRL-TO face challenges optimizing multiple objectives effectively. HDRL-PTO outperforms them in acceptance ratio and energy use. Although DRL-PTO uses the least energy in Network I, its lower acceptance ratio results in a lower episode reward than HDRL-PTO. DRL-TO shows significantly lower acceptance and rewards compared to HDRL-PTO in (Figs. 7, 5). Effective generalization ensures trained policies perform well across varied task parameters, crucial for real-world deployment. The proposed HDRL-PTO scheme demonstrates superior generalizability, as shown in Figs. 11 and 12, by successfully applying learned knowledge to new tasks. For experimental evaluation (Fig. 11 and 12, 100 tests were conducted on trained HL and LL agents to compare HDRL-PTO, DRL-PTO, and DRL-TO across tasks with diverse data sizes, computational demands, and

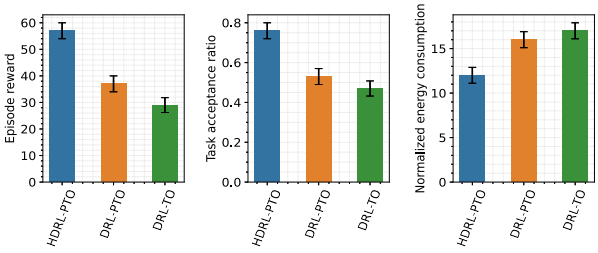


Fig. 11. Network I: Generalization performance over 100 experiments using trained models from three schemes.

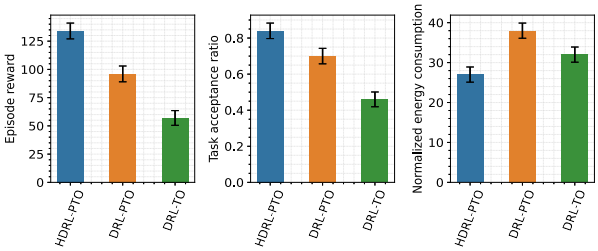


Fig. 12. Network II: Generalization performance over 100 experiments using trained models from three schemes.

latency constraints. Both familiar and novel tasks were used to assess agent generalization. Performance was measured via episode rewards, task acceptance ratios, and energy consumption, averaged over 100 runs. Confidence intervals at 80% and 95% levels were computed to evaluate result reliability. Figures 11 and 12 show results for Network I and II, respectively. Overall, HDRL-PTO outperforms others in generalization, confirming its adaptability to dynamic task environments in edge-cloud systems.

VII. Conclusion

In this study, we proposed the HDRL-PTO algorithm for dynamic edge–cloud IoT environments and compared its performance against DRL-PTO and DRL-TO. Simulation results demonstrated that HDRL-PTO consistently achieved higher episode rewards, with task acceptance ratios of 80–90% in both Network I and II, compared to 55–65% for DRL-PTO and 40–45% for DRL-TO. HDRL-PTO also reduced energy consumption while maintaining high task acceptance, outperforming DRL-PTO, which consumed more energy due to fixed partial offloading, and DRL-TO, which under-utilized resources. The hierarchical structure of HDRL-PTO, with HL and LL agents, enabled optimal offloading decisions, improved load balancing, and enhanced learning efficiency. Generalization tests across 100 experiments confirmed HDRL-PTO's adaptability to varying task sizes, computational demands, and latency constraints.

For future research, we intend to investigate the integration of Transformer-based Quantum DRL, meta-learning, and digital twin computing to further enhance adaptive decision-making, predictive task offloading, and scalable performance in large-scale IoT deployments. These advanced techniques are expected to improve resource allocation efficiency and robustness under increasingly dynamic network conditions.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] M. K. Mondal, S. Banerjee, D. Das, U. Ghosh, M. S. Al-Numay, and U. Biswas, "Toward energy-efficient and cost-effective task offloading in mobile edge computing for intelligent surveillance systems," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4087–4094, Feb. 2024.

[2] H. Jiang, X. Dai, Z. Xiao, and A. K. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4000–4015, Jul. 2023.

[3] T. Liu, D. Guo, Q. Xu, H. Gao, Y. Zhu, and Y. Yang, "Joint task offloading and dispatching for mec with rational mobile devices and edge nodes," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3262–3273, Jul.–Sep. 2023.

[4] X. Huang, S. Leng, S. Maharjan, and Y. Zhang, "Multi-agent deep reinforcement learning for computation offloading and interference coordination in small cell networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9282–9293, Sep. 2021.

[5] M. Teng, X. Li, and K. Zhu, "Joint optimization of sequential task offloading and service deployment in end-edge-cloud system for energy efficiency," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 283–298, May/Jun. 2024.

[6] Y. Chen, J. Zhao, Y. Wu, J. Huang, and X. S. Shen, "QoE-aware decentralized task offloading and resource allocation for end-edge-cloud systems: A game-theoretical approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 769–784, Jan. 2024.

[7] G. G. Devarajan, S. Thangam, M. J. F. Alenazi, U. Kumaran, G. Chandran, and A. K. Bashir, "Federated learning and blockchain-enabled framework for traffic rerouting and task offloading in the Internet of Vehicles (IoV)," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 3817–3825, May 2025.

[8] F. Liu, J. Huang, and X. Wang, "Joint task offloading and resource allocation for device-edge-cloud collaboration with subtask dependencies," *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 3027–3039, Jul.–Sep. 2023.

[9] X. Li, Z. Xu, F. Fang, Q. Fan, X. Wang, and V. C. Leung, "Task offloading for deep learning empowered automatic speech analysis in mobile edge-cloud computing networks," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1985–1998, Apr.–Jun. 2023.

[10] F. Zhang, G. Han, L. Liu, M. Martinez-Garcia, and Y. Peng, "Deep reinforcement learning based cooperative partial task offloading and resource allocation for IIoT applications," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2991–3006, Sep./Oct. 2023.

[11] K. Shuai, Y. Miao, K. Hwang, and Z. Li, "Transfer reinforcement learning for adaptive task offloading over distributed edge clouds," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 2175–2187, Apr.–Jun. 2023.

[12] Y. He, Y. Wang, Q. Lin, and J. Li, "Meta-hierarchical reinforcement learning (MHRL)-based dynamic resource allocation for dynamic vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 4, pp. 3495–3506, Apr. 2022.

[13] A. Younis, S. Maheshwari, and D. Pompili, "Energy-latency computation offloading and approximate computing in mobile-edge computing networks," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3401–3415, Jun. 2024.

[14] X. Xu et al., "Joint task offloading and resource optimization in noma-based vehicular edge computing: A game-theoretic DRL approach," *J. Syst. Archit.*, vol. 134, Jan. 2023, Art. no. 102780.

[15] Z. Xue, C. Liu, F. Wen, and G. Han, "Joint optimization of task offloading and resource allocation for cooperative perception in vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, early access, Jul. 8, 2025, doi: 10.1109/TVT.2025.3586857.

[16] C. Zeng, X. Wang, R. Zeng, Y. Li, J. Shi, and M. Huang, "Joint optimization of multi-dimensional resource allocation and task offloading for QoE enhancement in cloud-edge-end collaboration," *Future Gener. Comput. Syst.*, vol. 155, pp. 121–131, Jun. 2024.

[17] B. Wang, B. Lv, and Y. Song, "A hybrid genetic algorithm with integer coding for task offloading in edge-cloud cooperative computing," *IAENG Int. J. Comput. Sci.*, vol. 49, no. 2, p. 503, 2022.

[18] J. Zhai, J. Bi, H. Yuan, J. Zhang, and R. Buyya, "Energy-efficient and latency-aware task offloading for industrial cloud-edge systems with heterogeneous CPUs and GPUs," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 25757–25772, Jul. 2025.

[19] H. Xiao, J. Huang, Z. Hu, M. Zheng, and K. Li, "Collaborative cloud-edge-end task offloading in mec-based small cell networks with distributed wireless backhaul," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4542–4557, Dec. 2023.

[20] J. Chen, H. Wu, R. Li, and P. Jiao, "Green parallel online offloading for DSCI-type tasks in IoT-edge systems," *IEEE Trans. Ind. Informat.*, vol. 18, no. 11, pp. 7955–7966, Nov. 2022.

[21] L. Wu et al., "DOT: Decentralized offloading of tasks in OFDMA-based heterogeneous computing networks," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 20071–20082, Oct. 2022.

[22] S. Zhou, A. Ali, A. Al-Fuqaha, M. Omar, and L. Feng, "Robust risk-sensitive task offloading for edge-enabled Industrial Internet of Things," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 1403–1413, Feb. 2024.

[23] R. Zhang, P. Cheng, Z. Chen, S. Liu, B. Vucetic, and Y. Li, "Calibrated bandit learning for decentralized task offloading in ultra-dense networks," *IEEE Trans. Commun.*, vol. 70, no. 4, pp. 2547–2560, Apr. 2022.

[24] S. Wu, N. Chen, G. Wen, L. Xu, P. Zhang, and H. Zhu, "Virtual network embedding for task offloading in IIoT: A DRL-assisted federated learning scheme," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6814–6824, Apr. 2024.

[25] H. Lu, C. Gu, F. Luo, W. Ding, and X. Liu, "Optimization of lightweight task offloading strategy for mobile edge computing based on deep reinforcement learning," *Future Gener. Comput. Syst.*, vol. 102, pp. 847–861, Jan. 2020.

[26] L. Huang, X. Feng, C. Zhang, L. Qian, and Y. Wu, "Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing," *Digit. Commun. Netw.*, vol. 5, no. 1, pp. 10–17, 2019.

[27] W. Fan, "Hybrid deep reinforcement learning-based task offloading for D2D-assisted cloud-edge-device collaborative networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13455–13471, Dec. 2024.

[28] T. Li, Y. Liu, T. Ouyang, H. Zhang, K. Yang, and X. Zhang, "Multi-hop task offloading and relay selection for IoT devices in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 466–481, Jan. 2025.

[29] H. Byeon et al., "Consumer technology in task offloading and edge resource allocation: AIoT and edge computing for next-generation communication," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 5356–5365, May 2025.

[30] N. Sharma, A. Ghosh, R. Misra, and S. K. Das, "Deep meta Q-learning based multi-task offloading in edge-cloud systems," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2583–2598, Apr. 2024.

[31] C. Liu, F. Tang, Y. Hu, K. Li, Z. Tang, and K. Li, "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1603–1614, Jul. 2021.

[32] M. Chen, W. Liu, T. Wang, S. Zhang, and A. Liu, "A game-based deep reinforcement learning approach for energy-efficient computation in MEC systems," *Knowl. Based Syst.*, vol. 235, Jan. 2022, Art. no. 107660.

[33] J. Chen and Z. Wu, "Dynamic computation offloading with energy harvesting devices: A graph-based deep reinforcement learning approach," *IEEE Wireless Commun. Lett.*, vol. 25, no. 9, pp. 2968–2972, Sep. 2021.

[34] X. Qiu, W. Zhang, W. Chen, and Z. Zheng, "Distributed and collective deep reinforcement learning for computation offloading: A practical perspective," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1085–1101, May 2020.

[35] F. Chai, Q. Zhang, H. Yao, X. Xin, R. Gao, and M. Guizani, "Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite IoT," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7783–7795, Jun. 2023.

[36] Y. Dai, K. Zhang, S. Maharjan, and Y. Zhang, "Deep reinforcement learning for stochastic computation offloading in digital twin networks," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4968–4977, Jul. 2021.

[37] S. Chen, X. Ge, Q. Wang, Y. Miao, and X. Ruan, "DDPG-based intelligent rechargeable fog computation offloading for IoT," *Wireless Netw.*, vol. 28, no. 7, pp. 3293–3304, Oct. 2022.

[38] J. Zou, T. Hao, C. Yu, and H. Jin, "A3C-DO: A regional resource scheduling framework based on deep reinforcement learning in edge scenario," *IEEE Trans. Comput.*, vol. 70, no. 2, pp. 228–239, Feb. 2021.

[39] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11158–11168, Nov. 2019.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.

[40] H. Mashal and M. H. Rezvani, "Multiobjective offloading optimization in fog computing using deep reinforcement learning," *J. Comput. Netw. Commun.*, vol. 9, no. 1, 2024, Art. no. 6255511.
[41] W. Jiang, D. Feng, Y. Sun, G. Feng, Z. Wang, and X.-G. Xia, "Joint computation offloading and resource allocation for D2D-assisted mobile edge computing," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 1949–1963, May/Jun. 2023.
[42] L. Tan, Z. Kuang, L. Zhao, and A. Liu, "Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1960–1972, Mar. 2022.
[43] C. You, K. Huang, H. Chae, and B.-H. Kim, "Energy-efficient resource allocation for mobile-edge computation offloading," *IEEE Trans. Wireless Commun.*, vol. 16, no. 3, pp. 1397–1411, Mar. 2017.
[44] S. Ruder, "An overview of gradient descent optimization algorithms," 2016, *arXiv:1609.04747*.



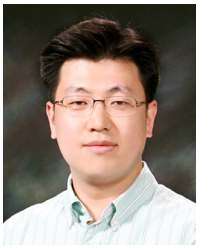
Ihsan Ullah received the B.Sc. and M.Sc. degrees in computer science from the University of Peshawar, Pakistan, in 2001 and 2004, respectively, and the Ph.D. degree in computer engineering from Sungkyunkwan University, Suwon, South Korea, in 2019, where he focused on intelligent network optimization and edge-cloud architectures. From 2019 to 2020, he was a Postdoctoral Research Fellow with the Ubiquitous Computing Technology Research Institute, Sungkyunkwan University, where he contributed to smart sensing and adaptive resource management. From 2020 to 2024, he was a Research Professor with the School of Computer Science and Engineering, Korea University of Technology and Education, Cheonan, South Korea, working on digital twin modeling and 5G network slicing. He is currently an Assistant Professor with the Department of Computer Science, Yuan Ze University, Taiwan, where he teaches and conducts research on intelligent network systems. His primary focus is on artificial intelligence, particularly reinforcement learning. He has led several research projects applying and improving reinforcement learning algorithms in areas, such as intelligent networking. His research interests also include data aggregation and fusion, virtual network embedding, digital twin frameworks, network slicing, the Internet of Things, deep reinforcement learning, graph neural networks, machine learning, cloud-edge computing, and wireless sensor networks.



Hyun-Kyo Lim received the B.S. degree in computer science and engineering and the M.S. degree in computer science engineering from the Korea University of Technology and Education, Cheonan, South Korea, in 2015 and 2017, respectively, and the Ph.D. degree from the Department of Interdisciplinary Program in Creative Engineering, Korea University of Technology and Education in 2022. Since 2022, he has been a Postdoctoral Researcher with the Future Convergence Engineering, Korea University of Technology and Education. He studied mobility management during his master's course and he especially researched distributed mobility management in software-defined networking. He is studying deep learning and reinforcement learning during his doctoral and postdoctoral studies. He is also exploring ways to apply deep learning and reinforcement learning to the network and is working on applying deep learning and reinforcement learning to a variety of applications.



Yeong-Jun Seok received the B.S. and M.S. degrees in computer science and engineering from the Korea University of Technology and Education, Cheonan, South Korea, in 2022 and 2024, respectively. He is currently pursuing the Ph.D. degree in computer science and engineering with the Korea University of Technology and Education. Since 2022, he has been continuously conducting research on applying deep reinforcement learning to various fields, such as task offloading and combinatorial optimization problems. Since 2024, he has been focusing on applying deep reinforcement learning to quantum communication and networks.



Youn-Hee Han (Member, IEEE) received the B.S. degree in mathematics, and the M.S. and Ph.D. degrees in computer science and engineering from Korea University, Seoul, South Korea, in 1996, 1998, and 2002, respectively. From 2002 to 2006, he was a Senior Researcher with the Next Generation Network Group, Samsung Advanced Institute of Technology, Suwon, South Korea. Since 2006, he has been a Professor with the School of Computer Science and Engineering, Korea University of Technology and Education, Cheonan, South Korea. He also served as a Visiting Professor with the Department of Computer Science, State University of New York at Albany, Albany, NY, USA, from September 2013 to January 2015. Since 2002, his activities have focused on mobility management, media-independent handover, and cross-layer optimization for efficient mobility support. He has published approximately 270 research articles on the theory and application of mobile computing and has led 45 patents regarding the information and communication technology domain. He is currently very interested in artificial intelligence technology, especially reinforcement learning, and he has been in charge of many research projects regarding improving the performance of reinforcement learning algorithms for various fields, such as intelligent networking on 5G and 6G, robotics, drones, economics, and financial engineering. He has made several contributions to IETF and IEEE standardization. He actively participated in the IEEE 802.21 Working Group, and was also an author of IETF RFC 5181, RFC 5270, and RFC 7864. His current research interests include the theory and application of computer networks, including protocol design and mathematical analysis, mobile sensor/actuator networks, social network analysis, machine learning, deep learning, and reinforcement learning.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on October 20,2026 at 13:28:21 UTC from IEEE Xplore. Restrictions apply.